Coffer MCP — Architecture and Security Review

Version: 0.2.0 | Author: Anna Hix | Date: March 19, 2026
Repository: github.com/annawhooo/coffer-mcp | Classification: Internal — For Security Review

1. Executive Summary

Coffer is an open-source Python MCP (Model Context Protocol) server that provides encrypted credential storage and authenticated request proxying for LLM agents. It enables AI assistants like Claude to make authenticated HTTP requests and access login-protected web portals without the credential ever appearing in the LLM's conversation context.

Key security properties:

- Credentials encrypted at rest with AES-256-GCM (per-entry unique nonces)
- Master key stored in OS keyring with random per-user PBKDF2 salt
- URL allowlisting with strict domain matching prevents credential use against unauthorized endpoints (fail-closed)
- Per-hop redirect checking against URL allowlist prevents open-redirect credential theft
- Response sanitization scrubs leaked credentials from bodies and error messages
- Prompt injection defense strips HTML comments, hidden elements, and invisible unicode from responses
- Browser sessions auto-expire after 30 minutes with URL allowlist enforcement on every navigation
- HMAC-SHA-256 audit chain keyed to master key (file-only attackers cannot forge entries)
- Credential expiry with automatic enforcement and expiring-soon warnings
- OAuth2 client_credentials with automatic token refresh
- Encrypted backup/restore for disaster recovery
- Credential health checks (coffer test) to verify credentials before use
- Credentials never cross into the LLM context window

2. Architecture Overview

2.1 Trust Boundaries

The system operates across four trust boundaries:

Boundary 1 — User-controlled (one-time setup). The user interacts with the Coffer CLI to initialize the master key and add credentials. Plaintext credentials exist only during this interactive session (masked input via getpass). The master key is derived via PBKDF2-HMAC-SHA256 (600,000 iterations, random 16-byte salt) and stored in the OS keyring.

Boundary 2 — Coffer MCP server process (localhost, STDIO). The MCP server runs as a local subprocess spawned by Claude Desktop. It communicates via STDIO (stdin/stdout JSON-RPC). All credential resolution, HTTP request injection, browser automation, and response sanitization happen inside this process. The server never listens on a network port.

Boundary 3 — LLM context (Claude Desktop). Claude Desktop communicates with Coffer via MCP tool calls over STDIO. Claude sends tool invocations (alias, URL, method) and receives sanitized results. Claude never receives passwords, API keys, tokens, session cookies, or usernames.

Boundary 4 — External targets (REST APIs, web portals). Authenticated requests are made from the Coffer server process to external targets. Credentials are injected into HTTP headers (for API proxy) or browser form fields (for web login). TLS handles in-transit encryption. HTTP redirects are checked per-hop against the URL allowlist before following.

2.2 Components

Component	Role	Technology
Encrypted store	Per-entry AES-256-GCM encryption	Python cryptography library
Keychain	Master key from OS keyring (random salt)	Python keyring library
URL allowlist	Strict domain + path enforcement	urlparse + fnmatch (path only)
Response sanitizer	Scrubs leaked creds from responses	String + base64 + URL-encoding
Content sanitizer	Strips prompt injection payloads	Regex (HTML comments, hidden elements, invisible unicode)
Audit logger	Append-only JSONL, HMAC-SHA-256 chain	JSON + hmac + hashlib
HTTP proxy tool	Injects auth headers, checks redirects	httpx (async, no auto-follow)
OAuth2 token mgr	Auto-fetch/cache/refresh OAuth2 tokens	httpx + in-memory cache
Credential test tool	Health check (pass/fail + latency)	httpx HEAD request
Web login tool	Headless browser form automation	Playwright (Chromium)
Web fetch tool	Authenticated content as markdown	Playwright + readability-lxml
Backup/restore	Encrypted export/import	AES-256-GCM with separate passphrase
MCP server	Registers tools, manages lifecycle	FastMCP over STDIO
CLI	Credential mgmt outside of Claude	Click

3. Credential Lifecycle

3.1 Ingestion (one-time, user-initiated)

1. User runs `coffer init` — enters master passphrase (masked via getpass)
2. Passphrase derived to 32-byte key via PBKDF2-HMAC-SHA256 (random 16-byte salt, 600K iterations per OWASP 2023)
3. Salt + derived key stored together as JSON in OS keyring (Windows Credential Manager)
4. User runs `coffer add [--expires 90d]` — enters credential details (alias, auth_type, username, secret, allowed_urls, allowed_methods, optional expiry)
5. Secret encrypted with AES-256-GCM using unique 12-byte random nonce (os.urandom)
6. Encrypted blob persisted to ~/.coffer/credentials.json via atomic file write (.tmp rename)
7. Plaintext secret exists only in process memory during steps 4-6

3.2 Usage — API Proxy Flow (coffer_http_request)

1. Claude calls coffer_http_request(alias, url, method)
2. Server decrypts the credential from the encrypted store
3. **Expiry check:** if credential has expired, returns error with rotation instructions
4. **URL allowlist:** checks URL against credential's strict allowlist (exact domain, fnmatch path). Fail-closed.
5. **Method allowlist:** checks HTTP method against allowed methods
6. If any check fails: returns error to Claude, logs denial in audit
7. **Auth injection:** injects appropriate header (Bearer, Basic, custom, or OAuth2 auto-token)
8. Makes HTTP request via httpx with **follow_redirects=False**
9. **Per-hop redirect check:** each redirect location verified against allowlist before following (max 10 hops)
10. **Credential sanitization:** scrubs plaintext secret, URL-encoded secret, base64 basic auth from response
11. **Content sanitization:** strips HTML comments, hidden elements, invisible unicode, truncates at 200K chars
12. Returns sanitized response to Claude
13. Logs access in HMAC-authenticated audit trail

3.3 Usage — Web Login Flow (coffer_web_login + coffer_web_fetch)

1. Claude calls coffer_web_login(alias, login_url, selectors...)
2. Server decrypts the credential, verifies auth_type is 'web_login'
3. Playwright launches headless Chromium, navigates to login page
4. Server fills username and password into form fields (CSS selectors)
5. Server clicks submit, waits for navigation
6. Authenticated browser context cached server-side with **30-minute expiry timestamp**
7. Returns { status: 'ok', page_title: '...' } to Claude — no credential data
8. Claude calls coffer_web_fetch(alias, url)
9. **Session expiry check:** if session older than 30 minutes, auto-closed with re-login prompt

10. **URL allowlist check:** fetch target verified against credential's allowlist
11. Server navigates authenticated browser to URL
12. Extracts main content via readability-lxml, converts to markdown
13. Content sanitization applied (prompt injection defense)
14. Sanitized markdown returned to Claude

4. Encryption Details

Property	Value
Algorithm	AES-256-GCM (authenticated encryption)
Key size	256 bits (32 bytes)
Nonce size	96 bits (12 bytes), unique per entry
Nonce generation	os.urandom(12) — CSPRNG
Key derivation	PBKDF2-HMAC-SHA256, 600,000 iterations
KDF salt	Random 16-byte (keyring path) or SHA-256(service_name)[:16] (env var fallback)
GCM tag	128 bits (included in ciphertext)
Key storage (preferred)	OS keyring — salt + key stored as JSON
Key storage (env var)	COFFER_MASTER_KEY env var (deterministic salt — weaker)
Key storage (fallback)	~/.coffer/.master-key (icacls on Windows, chmod 0600 on Unix)
Backup encryption	Separate AES-256-GCM with random salt + user-provided passphrase

Each credential is encrypted independently with its own nonce. The store uses atomic file writes via .tmp file rename to prevent corruption. Secret rotation (coffer rotate) is atomic: read-modify-write in a single _write_blobs() call with no window where the credential is missing from disk.

5. URL Allowlisting

Each credential defines which URLs it can be used against. The allowlist uses **strict matching**: scheme and domain (netloc) must match exactly — no wildcard subdomains. Only the path component supports fnmatch-style wildcards. Path traversal attempts (/../) are normalized before matching.

HTTP redirect defense: Coffer does not use httpx's follow_redirects=True. Instead, each redirect hop is manually followed with the destination URL checked against the allowlist before following. This prevents open-redirect attacks where a trusted domain redirects credentials to an attacker-controlled endpoint. Maximum 10 redirect hops.

Browser fetch defense: coffer_web_fetch enforces the URL allowlist on every navigation, not just the initial login. This prevents prompt injection attacks that could navigate an authenticated session to account settings, password change pages, or admin panels.

6. Audit System

Append-only JSONL at ~/.coffer/audit.jsonl. Each event includes: event_id, event_type, alias, status, details, timestamp, prev_hash, hash. The hash chain uses **HMAC-SHA-256 keyed to the master key**, so an attacker who gains file access but not the master key cannot recompute valid hashes after tampering. The `coffer audit` command verifies integrity.

Never logged: passwords, API keys, tokens, cookies, usernames, encrypted blobs.

7. Response Content Safety

Response bodies from target servers pass through two sanitization layers before reaching the LLM:

- **Credential scrubbing** (sanitize_response): Replaces the credential secret, its URL-encoded form, base64-encoded form, and auth headers with [REDACTED]. Also applied to httpx exception messages.
- **Content safety** (sanitize_content): Strips HTML comments, hidden HTML elements (display:none, visibility:hidden, opacity:0), zero-width/invisible Unicode characters, and truncates oversized responses at 200,000 characters.

8. Credential Expiry and Health Checks

Credentials support an optional `expires_at` timestamp. When set:

- `coffer list` shows EXPIRED or EXPIRING_SOON (within 7 days) status
- Expired credentials are blocked from use — Claude gets a clear error with rotation instructions
- `coffer test` verifies credentials work with a lightweight HEAD request, reporting pass/fail, HTTP status code, and latency in milliseconds
- `coffer rotate` atomically updates the secret with confirmation

9. Threat Model

9.1 Threats Mitigated

Threat	Mitigation
LLM context leakage	Credentials resolved server-side; only sanitized results returned
Plaintext credential storage	AES-256-GCM encryption at rest with per-entry nonces
Prompt injection via response	HTML comments, hidden elements, invisible unicode stripped; 200K char truncation
Prompt injection exfiltration	Strict URL allowlisting (exact domain, fnmatch path only)
Open-redirect credential theft	Per-hop redirect checking against URL allowlist
Credential leak in errors	Exception messages sanitized before returning to LLM
Credential enumeration	coffer_list returns aliases only, never secrets
Audit log tampering	HMAC-SHA-256 chain keyed to master key
Browser session hijack	30-min auto-expiry + URL allowlist on every fetch
Race condition cred loss	Atomic update_secret (single file write)
Rainbow table attacks	Random 16-byte PBKDF2 salt per user (keyring path)
Windows .master-key exposure	icacls restricts to current user (replaces no-op chmod)
Expired credential use	Expiry enforced in vault_http_request before any network call

9.2 Known Gaps (Threats NOT Mitigated)

Threat	Notes	Mitigation Path
Compromised host	Attacker can read process memory	Full-disk encryption; keep OS patched
Master key in process memory	No mlock/VirtualLock in current version	v2: memory protection
Playwright browser memory	Plaintext exists briefly in Chromium	Sessions auto-expire (30 min)
Malicious MCP client	Client ignoring output boundaries	Use only trusted MCP clients
Supply-chain attack	Modified package could change tool behavior	Pin versions; verify checksums
MCP stdio MITM	No mutual auth on local IPC channel	MCP protocol limitation; host security
Env var path (weak salt)	Deterministic salt for COFFER_MASTER_KEY	Prefer coffer init (random salt)
Novel prompt injection	Content sanitizer may miss new techniques	URL allowlist is primary defense
Audit log deletion	HMAC detects modification but not truncation	External log forwarding (v2)

10. Data Flow Summary

Crosses from Coffer to Claude (safe):

Credential aliases and metadata, expiry status, test results (pass/fail + latency), sanitized HTTP response bodies, status codes, web page content as markdown, login status messages, audit events, error messages (sanitized).

NEVER crosses from Coffer to Claude:

Passwords, API keys, tokens, session cookies, usernames, encrypted blobs, nonces, master key material, raw HTTP auth headers, OAuth2 client secrets, backup passphrases.

11. File Layout

- `~/.coffer/credentials.json` — Encrypted credential entries (AES-256-GCM)
- `~/.coffer/audit.jsonl` — Append-only audit log with HMAC chain
- `~/.coffer/.master-key` — Auto-generated master key (fallback only, icacls/0600)

12. Dependencies

Package	Purpose	Notes
mcp >= 1.0.0	MCP protocol SDK	Anthropic-maintained
cryptography >= 42.0.0	AES-256-GCM	Widely audited, OpenSSL-backed
keyring >= 25.0.0	OS keyring access	Platform-specific backends
httpx >= 0.27.0	Async HTTP client	API proxy + OAuth2 token fetch
playwright >= 1.40.0	Browser automation	Microsoft-maintained (optional)
readability-lxml >= 0.8.1	Content extraction	Mozilla algorithm port
html2text >= 2024.2.26	HTML to markdown	Lightweight
click >= 8.1.0	CLI framework	Pallets project

13. Recommendations for Hardening (v2+)

1. Memory protection — mlock/VirtualLock to prevent master key swap to disk
2. Rate limiting — per-alias limits on tool invocations to mitigate prompt injection spam
3. MFA/TOTP support — handle 2FA challenges during web login
4. Credential scoping — time-of-day and IP restrictions for API proxy
5. External audit log forwarding — SIEM integration for tamper detection beyond local HMAC
6. Multi-user shared vault — team credential access with per-user keys
7. Approval workflows — native OS confirmation prompts for high-sensitivity credentials
8. Plugin architecture for auth types — custom auth handlers (SAML, mTLS, AWS SigV4)
9. Formal security audit — third-party penetration testing